ConFluent Kafka のベストプラクティスをご確認ください

NetApp Solutions

NetApp
March 19, 2024

This PDF was generated from https://docs.netapp.com/ja-jp/netapp-solutions/data-analytics/confluent-kafka-introduction.html on March 19, 2024. Always check docs.netapp.com for the latest.

目次

ConFluent Kafka のベストプラクティスをご確認ください . . . 1
TR-4912 ：『 Best Practices guidelines for ConFluent Kafka Tiered Storage with NetApp 』 . . . 1
解決策アーキテクチャの詳細 . . . 3
テクノロジの概要 . . . 4
矛盾する検証 . . . 11
拡張性を備えたパフォーマンステスト . . . 14
矛盾点 3 コネクタ . . . 16
競合する自己バランシングクラスタ . . . 25
ベストプラクティスのガイドライン . . . 25
サイジング . . . 26
まとめ . . . 30

ConFluent Kafka のベストプラクティスをご確認ください

TR-4912 ：『 Best Practices guidelines for ConFluent Kafka Tiered Storage with NetApp 』

Karthikeyan Nagalingam 、 Joseph Kandatilparamil 、 NetApp Rankesh Kumar 、 Conluent

Apache Kafka は、 1 日に数兆ものイベントを処理できる、コミュニティで分散されたイベントストリーミングプラットフォームです。Kafka は、当初はメッセージングキューとして構築され、分散コミットログの抽象化に基づいています。Kafka は、 2011 年に LinkedIn で作成されオープンソースとなって以来、メッセージキューから本格的なイベントストリーミングプラットフォームへと進化してきました。Confluent Platform で Apache Kafka を配布します。Coneluent Platform は、 Kafka を補完するための追加のコミュニティ機能と商用機能を備えています。これらの機能は、本番環境の運用者と開発者の両方のストリーミングエクスペリエンスを大規模に向上させるように設計されています。

本ドキュメントでは、次のコンテンツを提供することで、ネットアップのオブジェクトストレージ製品での上位階層型ストレージの使用に関するベストプラクティスのガイドラインについて説明します。

- ネットアップオブジェクトストレージとの競合検証– NetApp StorageGRID
- 階層型ストレージのパフォーマンステスト
- ネットアップのストレージシステムを使用する場合のベストプラクティスのガイドラインを参照してください

階層型ストレージが優れている理由

競合製品は、多くのアプリケーション、特にビッグデータ、分析、ストリーミングワークロードに対応するデフォルトのリアルタイムストリーミングプラットフォームになっています。階層型ストレージを使用すると、ユーザは Conluent プラットフォームのストレージからコンピューティングを分離できます。データをより対費用効果の高い方法で保存し、ほぼ無制限のデータ量を保存し、オンデマンドでワークロードを増減できます。また、データやテナントのリバランシングなどの管理タスクが容易になります。S3 互換のストレージシステムでは、これらすべての機能を活用して、すべてのイベントのデータを 1 箇所で民主化できるため、複雑なデータエンジニアリングは不要です。Kafka に階層化ストレージを使用すべき理由については、を参照してください "この記事は流暢なものです"。

階層化ストレージに **NetApp StorageGRID** を使用する理由

StorageGRID は、業界をリードするネットアップのオブジェクトストレージプラットフォームです。StorageGRID は、ソフトウェアで定義されるオブジェクトベースのストレージ解決策で、 Amazon Simple Storage Service （ S3 ） API などの業界標準のオブジェクト API をサポートします。StorageGRID は、大規模な非構造化データを格納および管理し、セキュアでデータ保持性に優れたオブジェクトストレージを実現します。コンテンツは適切なタイミングで適切な場所の適切なストレージ階層に配置されるため、グローバルに分散されるリッチメディアのワークフローを最適化し、コストを削減できます。

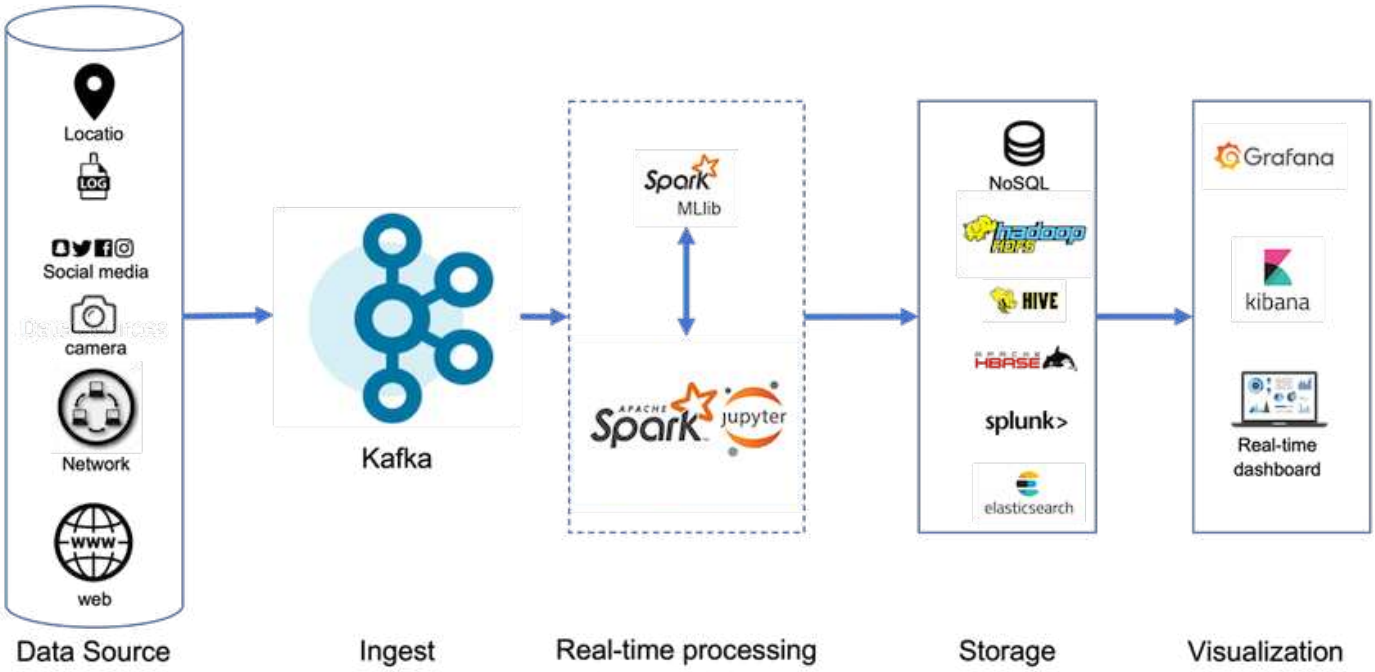
StorageGRID の最大の差別化要因は、ポリシーベースのデータライフサイクル管理を可能にする情報ライフサイクル管理（ ILM ）ポリシーエンジンです。ポリシーエンジンでは、メタデータを使用して、データの有効期間全体の格納方法を管理できます。これにより、最初はパフォーマンスを最適化し、データの経過に応じてコストと保持性を自動的に最適化できます。

階層型ストレージを有効にします

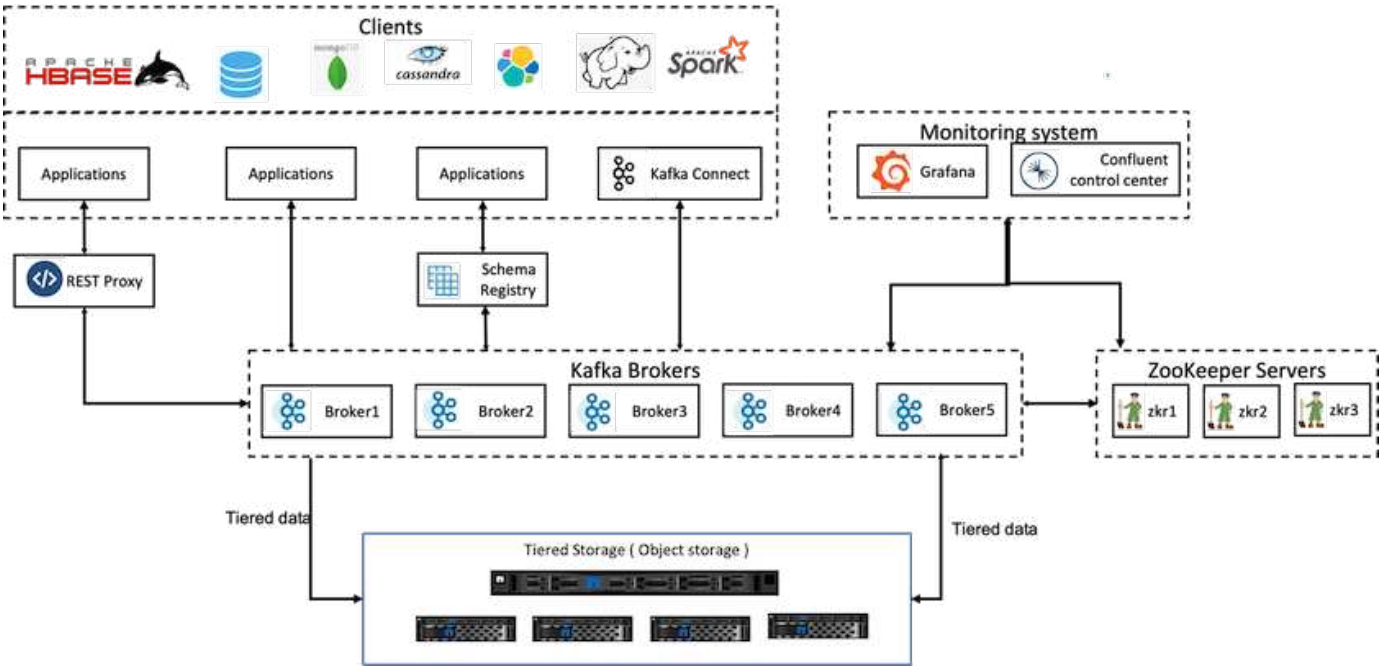
階層型ストレージの基本的な目的は、データストレージのタスクをデータ処理から分離することです。この分離によって、データストレージ階層とデータ処理階層を別々に拡張しやすくなります。

流暢な層ストレージの解決策は、 2 つの要因で競合する必要があります。まず、リスト操作の不整合やオブジェクトを使用できないことがあるなど、一般的なオブジェクトストアの整合性と可用性のプロパティを回避または回避する必要があります。次に、階層型ストレージと Kafka のレプリケーションとフォールトトレランスモデルの間の相互作用を正しく処理する必要があります。これには、ゾンビのリーダーが引き続きオフセット範囲を階層化する可能性も含まれます。ネットアップのオブジェクトストレージは、整合性のあるオブジェクトを使用できるようにするとともに、 HA モデルによって、オフセット範囲を階層化するために使用できるストレージにします。ネットアップのオブジェクトストレージを使用すると、オブジェクトの可用性に一貫性があり、オフセット範囲を階層化するために使用される階層化ストレージには HA モデルが採用されています。

階層型ストレージでは、ストリーミングデータの末尾付近での低レイテンシの読み取りや書き込みにハイパフォーマンスプラットフォームを使用できます。また、 NetApp StorageGRID などの低コストで拡張性に優れたオブジェクトストレージを使用して、高スループットの履歴読み取りを実行することもできます。また、ネットアップストレージコントローラを搭載した Spark に関するテクニカル解決策もご用意しています。詳細はこちらをご覧ください。Kafka がリアルタイムの分析パイプラインにどのように適しているかを次の図に示します。



次の図は、 NetApp StorageGRID が ConFluent Kafka のオブジェクトストレージ層にどのように適合するかを示しています。



解決策アーキテクチャの詳細

このセクションでは、流暢な検証に使用するハードウェアおよびソフトウェアについて説明します。この情報は、ネットアップストレージを使用した流暢なプラットフォームの導入に該当します。次の表に、テストに使用した解決策アーキテクチャと基本コンポーネントを示します。

解決策コンポーネント	詳細
Kafka バージョン 6.2 と競合します	• ご主人の 3 人 • 5 台のブローカーサーバ • 5 台のツールサーバ • Grafana × 1 • 1 つのコントロールセンター
Linux （ Ubuntu 18.04 ）	すべてのサーバ
階層化ストレージ向けの NetApp StorageGRID	• StorageGRID ソフトウェア • SG1000 × 1 （ロードバランサ） • SGF6024 × 4 • 24 本、 800 本の SSD × 4 • S3 プロトコル • 100GbE × 4 （ブローカーと StorageGRID インスタンス間のネットワーク接続）

解決策コンポーネント	詳細
Fujitsu Primergy RX2540 サーバ × 15	各モデルには、 CPU × 2 、物理コア × 16 、 Intel Xeon × 256GB 物理メモリ × 100GbE デュアルポートが搭載されています

テクノロジの概要

このセクションでは、この解決策で使用されるテクノロジーについて説明します。

NetApp StorageGRID

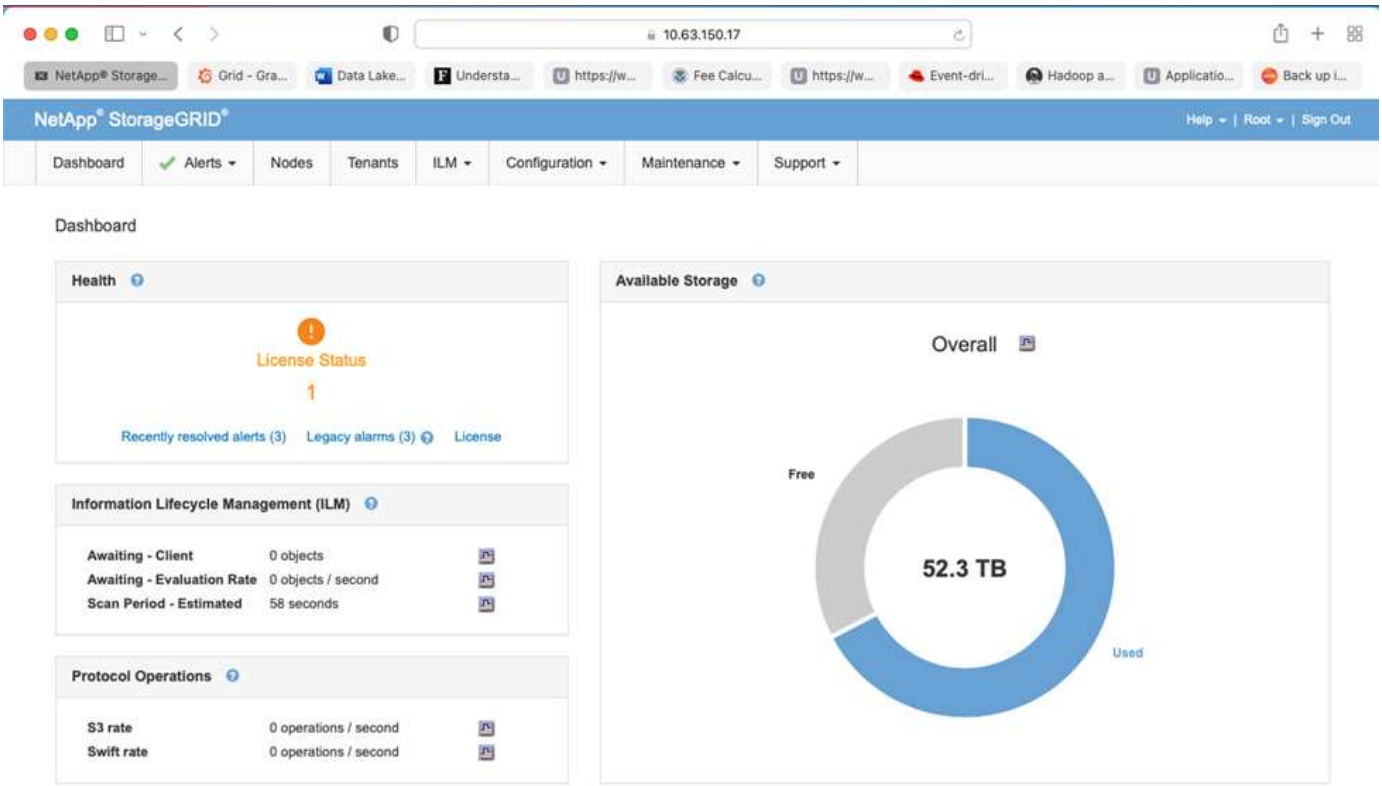
NetApp StorageGRID は、ハイパフォーマンスで対費用効果の高いオブジェクトストレージプラットフォームです。階層型ストレージを使用することで、ローカルストレージやブローカーの SAN ストレージに格納されている ConFluent Kafka にあるデータのほとんどが、リモートのオブジェクトストアにオフロードされます。この構成では、クラスタのリバランシング、拡張、縮小、障害が発生したブローカーの交換にかかる時間とコストを削減することで、運用が大幅に改善されます。オブジェクトストレージは、オブジェクトストア階層にあるデータの管理に重要な役割を果たします。そのため、適切なオブジェクトストレージを選択することが重要です。

StorageGRID は、ノードベースの分散グリッドアーキテクチャを使用して、インテリジェントでポリシーベースのグローバルデータ管理を実現します。数ペタバイトの非構造化データと数十億のオブジェクトを、ユビキタスなグローバルオブジェクトネームスペースと高度なデータ管理機能を組み合わせることでシンプルに管理できます。単一コールのオブジェクトアクセスは、サイト間を拡張し、高可用性アーキテクチャを簡素化しながら、サイトやインフラストラクチャの停止に関係なく、オブジェクトへの継続的なアクセスを保証します。

マルチテナンシーを使用すると、複数の非構造化クラウドアプリケーションやエンタープライズデータアプリケーションを同じグリッド内で安全に処理できるため、 NetApp StorageGRID の ROI とユースケースが向上します。メタデータベースのオブジェクトライフサイクルポリシーを使用して複数のサービスレベルを作成し、複数の地域にわたるデータの保持、保護、パフォーマンス、ローカリティを最適化できます。ユーザは、データ管理ポリシーを調整し、トラフィック制限を監視および適用して、絶えず変化する IT 環境で要件が変化した場合に備えて、システムを停止することなくデータランドスケープに再調整できます。

Grid Manager で管理を簡易化

StorageGRID Grid Manager はブラウザベースのグラフィカルインターフェイスで、世界中に分散された複数のサイトにまたがる StorageGRID システムの設定、管理、監視を、 1 つの画面で実行できます。



StorageGRID グリッドマネージャインターフェイスでは、次のタスクを実行できます。

- イメージ、ビデオ、レコードなどのオブジェクトを収めた、グローバルに分散されたペタバイト規模のリポジトリを管理します。
- グリッドノードとサービスを監視してオブジェクトの可用性を確保します。
- Information Lifecycle Management （ ILM ；情報ライフサイクル管理）ルールを使用してオブジェクトデータの配置を継続的に管理します。これらのルールによって、取り込まれたオブジェクトのデータの処理、損失から保護する方法、格納場所と保管期間が決まります。
- システム内のトランザクション、パフォーマンス、処理を監視します。

情報ライフサイクル管理ポリシー

StorageGRID には、オブジェクトのレプリカコピーを保持し、特定のパフォーマンスおよびデータ保護要件に応じて 2+1 や 4+2 などの EC （イレイジャーコーディング）スキームを使用してオブジェクトを格納するなどの、柔軟なデータ管理ポリシーが用意されています。ワークロードと要件が時間の経過とともに変化する場合、 ILM ポリシーも時間の経過とともに変化する必要があることがよくあります。ILM ポリシーの変更は中核的な機能であり、絶えず変化する環境に StorageGRID のお客様がすばやく簡単に適応できるようにします。を確認してください "ILM ポリシー" および "ILM ルール" StorageGRID でセットアップする。

パフォーマンス

StorageGRID は、ストレージノードを追加することでパフォーマンスを拡張します。ストレージノードには、 VM 、ベアメタル、またはなどの専用アプライアンスを指定できます "SG5712 、 SG5760 、 SG6060 、 SGF6024"。今回のテストでは、 SGF6024 アプライアンスを使用した最小サイズの 3 ノードグリッドで、 Apache Kafka の主要なパフォーマンス要件を超えました。Kafka クラスタを追加のブローカーとともに拡張すれば、ストレージノードを追加してパフォーマンスと容量を高めることができます。

ロードバランサとエンドポイントの設定

StorageGRID の管理ノードは、 StorageGRID システムを表示、設定、管理するための Grid Manager UI （ユーザインターフェイス）エンドポイントと REST API エンドポイント、およびシステムアクティビティを追跡するための監査ログを提供します。そこで、 Conluent Kafka の階層化ストレージに可用性の高い S3 エンドポイントを提供するために、 StorageGRID ロードバランサを実装しました。このロードバランサは、管理ノードとゲートウェイノードでサービスとして実行されます。また、ロードバランサはローカルトラフィックを管理し、ディザスタリカバリに役立つ GSLB （グローバルサーバロードバランシング）と通信します。

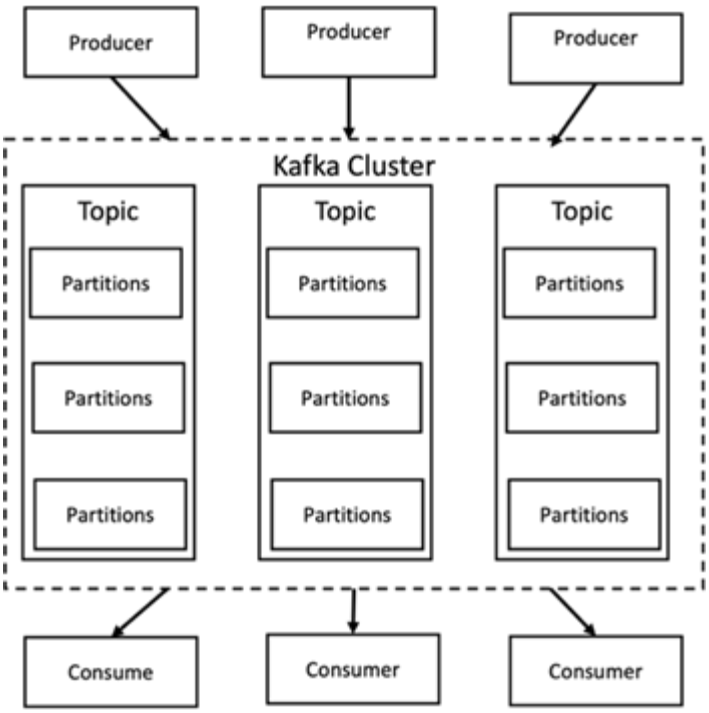
エンドポイントの設定をさらに強化するために、 StorageGRID は管理ノードに組み込まれたトラフィック分類ポリシーを提供し、ワークロードトラフィックを監視し、さまざまな Quality of Service （ QoS ；サービス品質）制限をワークロードに適用できます。トラフィック分類ポリシーは、ゲートウェイノードおよび管理ノードの StorageGRID ロードバランササービス上のエンドポイントに適用されます。これらのポリシーは、トラフィックシェーピングおよびモニタリングに役立ちます。

StorageGRID でのトラフィック分類

StorageGRID には QoS 機能が組み込まれています。トラフィック分類ポリシーを使用すると、クライアントアプリケーションからのさまざまなタイプの S3 トラフィックを監視できます。次に、ポリシーを作成して適用し、イン / アウト帯域幅、読み取り / 書き込み同時要求の数、または読み取り / 書き込み要求の速度に基づいて、このトラフィックに制限を設けることができます。

Apache Kafka です

Apache Kafka は、 Java と Scala で書かれたストリーム処理を使用したソフトウェアバスのフレームワーク実装です。リアルタイムデータフィードを処理するための、スループットが高く、低レイテンシの統合プラットフォームを提供することを目的としています。Kafka は外部システムに接続して Kafka Connect からデータをエクスポートし、インポートすることができます。また、 Java のストリーム処理ライブラリである Kafka ストリームが提供されます。Kafka では、効率性を重視して最適化されたバイナリの TCP ベースのプロトコルを使用しています。また、ネットワーク往復のオーバーヘッドを軽減するために、メッセージを自然にまとめてグループ化する「メッセージセット」抽象化に依存しています。これにより、より大規模なシーケンシャルディスク処理や、大容量のネットワークパケット、連続するメモリブロックが実現し、 Kafka では、バースト性の高いランダムメッセージ書き込みをリニア書き込みに変換することができます。次の図は、 Apache Kafka の基本的なデータフローを示しています。



Kafka には、 Producer と呼ばれる任意の数のプロセスから生成されるキーと値のメッセージが格納されます。データは、異なるトピック内の異なるパーティションにパーティショニングできます。パーティション内では、メッセージはオフセット（パーティション内のメッセージの位置）によって厳密に順序付けされ、インデックスが作成され、タイムスタンプとともに格納されます。コンシューマと呼ばれる他のプロセスは、パーティションからメッセージを読み取ることができます。Kafka はストリーム処理用の API を提供しており、Kafka からデータを利用する Java アプリケーションを作成して Kafka に結果を書き込むことができます。Apache Kafka は、 Apache Apex 、 Apache Flink 、 Apache Spark 、 Apache Storm 、 Apache NiFi などの外部ストリーム処理システムとも連携します。

Kafka は 1 つ以上のサーバで構成されたクラスタ（ブローカー）上で実行され、すべてのトピックのパーティションがクラスタノード全体に分散されます。さらに、パーティションは複数のブローカーにレプリケートされます。Kafka はこのアーキテクチャにより、フォールトトレラントな方法で大量のメッセージストリームを配信でき、 Java Message Service （ JMS ）や Advanced Message Queuing Protocol （ AMQP ）などの従来のメッセージングシステムの一部を置き換えることができます。0.11.0.0 リリース以降、 Kafka はトランザクション書き込みを提供しており、これは Streams API を使用して一度のストリーム処理を提供します。

Kafka では、 Regular とコンパクションの 2 種類のトピックをサポートしています。通常のトピックでは、保持期限またはスペースバインドを設定できます。指定した保持期限よりも古いレコードがある場合や、パーティションのスペースバインドを超過している場合、 Kafka では古いデータを削除してストレージスペースを解放することができます。デフォルトでは、トピックの保持期間は 7 日間に設定されていますが、データを無期限に保存することもできます。コンパクションの対象となるトピックについては、レコードの有効期限は時刻やスペースの上限に基づいて切れません。Kafka では、以降のメッセージを同じキーを持つ古いメッセージの更新として扱い、キーごとに最新のメッセージを削除しないことを保証しています。ユーザは、特定のキーのヌル値を持つ、いわゆる tombstone メッセージを書き込むことによって、メッセージを完全に削除できます。

Kafka には 5 つの主要な API があります。

- * Producer API. * は、アプリケーションがレコードのストリームをパブリッシュすることを許可します。
- *Consumer API. * は、アプリケーションがトピックを購読し、レコードのストリームを処理することを許可します。

- * Connector API. * は、トピックを既存のアプリケーションにリンクできる再利用可能なプロデューサおよびコンシューマ API を実行します。
- *Streams API. * この API は入力ストリームを出力に変換し、結果を生成します。
- * 管理者 API 。 Kafka のトピック、ブローカー、その他の Kafka のオブジェクトを管理するのに使用されます。

Kafka メッセージングプロトコルをベースに構築されたコンシューマ向け API とプロデューサー用 API は、Java で Kafka コンシューマクライアントとプロデューサークライアント向けのリファレンス実装を提供します。基本的なメッセージングプロトコルは、開発者が任意のプログラミング言語で独自のコンシューマクライアントまたはプロデューサクライアントを作成するために使用できるバイナリプロトコルです。これにより、Java Virtual Machine （ JVM ； Java 仮想マシン）エコシステムの Kafka のロックが解除されます。使用可能な Java 以外のクライアントの一覧は、 Apache Kafka wiki で管理されています。

Apache Kafka のユースケース

Apache Kafka は、メッセージング、 Web サイトのアクティビティ追跡、指標、ログ集約、ストリーム処理に最もよく使用されています。 イベントのソーシングとロギングのコミット

- Kafka はスループットの向上、組み込みのパーティショニング、レプリケーション、およびフォールトトレランスを実現しており、大規模なメッセージ処理アプリケーションに適した解決策となっています。
- Kafka では、リアルタイムのパブリッシュサブスクライブフィードのセットとして、追跡パイプラインでユーザのアクティビティ（ページビュー、検索）を再構築できます。
- Kafka は、多くの場合、運用監視データに使用されます。これには、分散アプリケーションからの統計情報を集約して、運用データの一元化フィードを作成する作業が含まれます。
- 多くの人が、ログアグリゲーション解決策の代わりに Kafka を使用しています。ログアグリゲーションは、一般にサーバから物理ログファイルを収集して処理のために一元的な場所（ファイルサーバや HDFS など）に配置します。Kafka は、ファイルの詳細を抽象化し、ログやイベントデータをメッセージのストリームとしてより明確に抽象化します。これにより、低レイテンシの処理が可能になり、複数のデータソースと分散データ消費のサポートが容易になります。
- Kafka のユーザの多くは、複数のステージで構成されるパイプラインでデータを処理しています。 Kafka のトピックから生の入力データが消費され、さらに消費やフォローアップ処理のために、集約、エンリッチ化、または新しいトピックへと変換されます。たとえば、ニュース記事を推薦するための処理パイプラインでは、 RSS フィードから記事のコンテンツをクロールし、それを「記事」トピックに公開することができます。さらに処理を行うと、このコンテンツをノーマライズまたは重複排除し、クレンジングされた記事コンテンツを新しいトピックにパブリッシュすることができます。また、最終的な処理段階では、このコンテンツをユーザーに推奨しようとする場合があります。このような処理パイプラインでは、個々のトピックに基づいてリアルタイムのデータフローのグラフが作成されます。
- イベントソースとは、状態の変化を時系列のレコードとしてログに記録するアプリケーション設計のスタイルです。Kafka は、非常に大容量の格納ログデータをサポートしているため、この形式のアプリケーションのバックエンドとして最適です。
- Kafka は分散システム用の一種の外部コミットログとして機能します。ログはノード間でデータをレプリケートするのに役立ち、障害が発生したノードがデータをリストアする際の再同期メカニズムとして機能します。Kafka のログコンパクション機能は、このユースケースに対応しています。

矛盾する

Conflicent Platform は、 Kafka を完成させるエンタープライズ対応プラットフォームです。高度な機能を備えており、アプリケーションの開発と接続を高速化し、ストリーム処理による変換を可能にし、大規模なエンタープライズ運用を簡易化し、厳しいアーキテクチャ要件に対応します。ConFluent では、 Apache Kafka を作

成した元のクリエイターが開発したサービスを利用して、 Kafka のメリットをエンタープライズクラスの機能で拡張しながら、 Kafka の管理や監視の負担を軽減することができます。現在、 Fortune 100 企業の 80% 以上がデータストリーミングテクノロジを採用しており、そのほとんどが Conluent 社を使用しています。

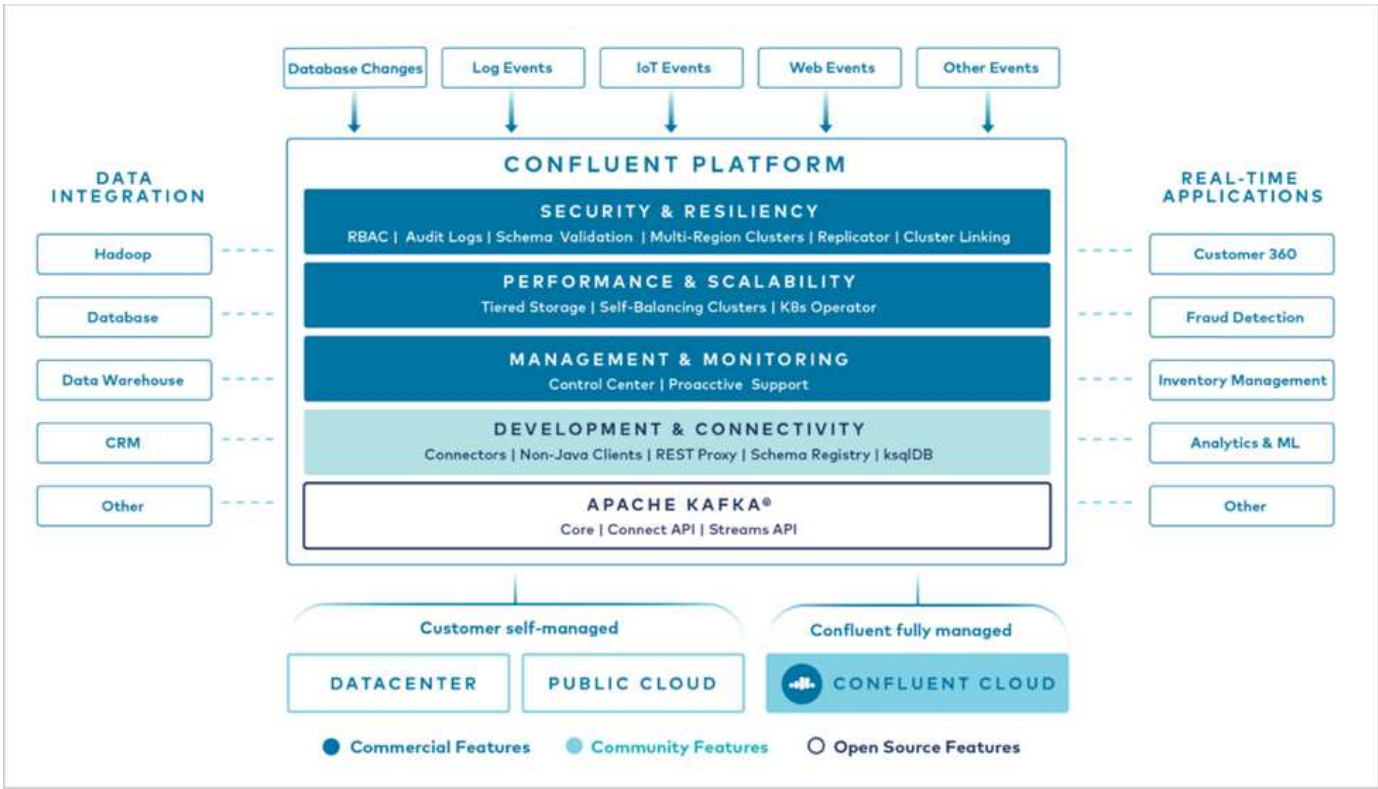
流暢な理由

履歴データとリアルタイムデータを一元化された単一の情報源に統合することで、 Conluent は、まったく新しいカテゴリの最新のイベント駆動型アプリケーションを簡単に構築し、ユニバーサルデータパイプラインを取得し、拡張性、パフォーマンス、信頼性を備えた強力な新しいユースケースを開放します。

流暢なものは何のために使用されるか。

Conflicent Platform を使用すると、データが異なるシステム間でどのように転送または統合されるかなど、基本的なメカニズムを気にすることなく、データからビジネス価値を引き出す方法に集中できます。具体的には、 Con裕福 なプラットフォームによって、 Kafka へのデータソースの接続やストリーミングアプリケーションの構築、 Kafka インフラの保護、監視、管理が簡易化されます。現在、 Conluent Platform は、金融サービス、オムニチャネル小売、自律走行車など、さまざまな業界のさまざまなユースケースに使用されています。 マイクロサービス、 IoT 。

以下の図は、 ConFluent Kafka Platform のコンポーネントを示しています。



流暢なイベントストリーミング技術の概要

流暢なプラットフォームの中核はです "Apache Kafka です"最も人気の高いオープンソースの分散ストリーミングプラットフォームです。Kafka の主な機能は次のとおりです。

- レコードのストリームをパブリッシュしてサブスクライブします。
- レコードのストリームをフォールトトレラントな方法で保存します。

- レコードのストリームを処理します。

Conluent Platform には Schema Registry 、 REST Proxy 、合計 100 以上の Kafka コネクタ、および ksqlDB も含まれています。

流暢なプラットフォームのエンタープライズ機能の概要

- * Conluent Control Center * Kafka を管理および監視するための GUI ベースのシステム。Kafka Connect の管理や、他のシステムとの接続の作成、編集、管理を簡単に行うことができます。
- * Kubernetes には流暢な言葉があります。 * Kubernetes の流暢な言葉は Kubernetes のオペレータです。Kubernetes の運用担当者は、特定のプラットフォームアプリケーションに固有の機能と要件を提供することで、 Kubernetes のオーケストレーション機能を拡張します。Con裕福 なプラットフォームの場合は、 Kubernetes での Kafka の導入プロセスを大幅に簡易化し、一般的なインフラのライフサイクルタスクを自動化します。
- * Kafka コネクタは、 Kafka Connect API を使用して、 Kafka をデータベース、キーバリューストア、検索インデックス、ファイルシステムなどの他のシステムに接続します。Confluent Hub には、一般的なデータソースおよびシンク用のダウンロード可能なコネクタがあります。これには、 Conluent Platform でこれらのコネクタの完全なテストとサポートされたバージョンが含まれます。詳細については、を参照してください "こちらをご覧ください"。
- * セルフバランシングクラスタ。 * 自動ロードバランシング、障害検出、自己修復機能を提供します。必要に応じてブローカーの追加や運用停止をサポートし、手動での調整は不要です。
- * クラスタを直接接続し、リンクブリッジを介して 1 つのクラスタから別のクラスタにトピックをミラーリングします。クラスタリンクにより、マルチデータセンター、マルチクラスタ、ハイブリッドクラウドの導入を簡易化できます。
- * 流暢な自動データバランサ。 * ブローカーの数、パーティションのサイズ、パーティションの数、およびクラスタ内のリーダーの数について、クラスタを監視します。これにより、データを移動してクラスタ全体で均等なワークロードを作成しながら、トラフィックのリバランシングを調整して、リバランシング中の本番ワークロードへの影響を最小限に抑えることができます。
- * 流暢なリプリケータ * により、複数のデータセンターで複数の Kafka クラスターを容易に保守できます。
- * 階層化ストレージ。 * 任意のクラウドプロバイダを使用して大量の Kafka データを保存するオプションを提供し、運用上の負担とコストを削減します。階層型ストレージでは、コスト効率に優れたオブジェクトストレージにデータを格納し、ブローカーを拡張するために、必要なコンピューティングリソースが増えた場合のみデータを利用できます。
- * Conluent JMS Client. * Conluent Platform には Kafka 用の JMS 対応クライアントが含まれています。Kafka クライアントは、 Kafka ブローカーをバックエンドとして使用して、 JMS 1.1 標準 API を実装しています。これは 'JMS を使用するレガシーアプリケーションがあり ' 既存の JMS メッセージブローカを Kafka に置き換える場合に便利です
- * Coneluent MQTT プロキシ * を使用すると、 MQTT デバイスやゲートウェイから Kafka に直接データを公開できます。 MQTT ブローカーは必要ありません。
- * 流暢なセキュリティプラグイン。 * 流暢なセキュリティプラグインは、各種の流暢なプラットフォームツールや製品にセキュリティ機能を追加するために使用されます。現在、 Conluent REST プロキシ用のプラグインが用意されており、受信要求の認証に役立ち、認証されたプリンシパルを要求に Kafka に伝播できます。これにより、 Con裕福 な REST プロキシクライアントでは、 Kafka ブローカーのマルチテナントセキュリティ機能を利用できます。

矛盾する検証

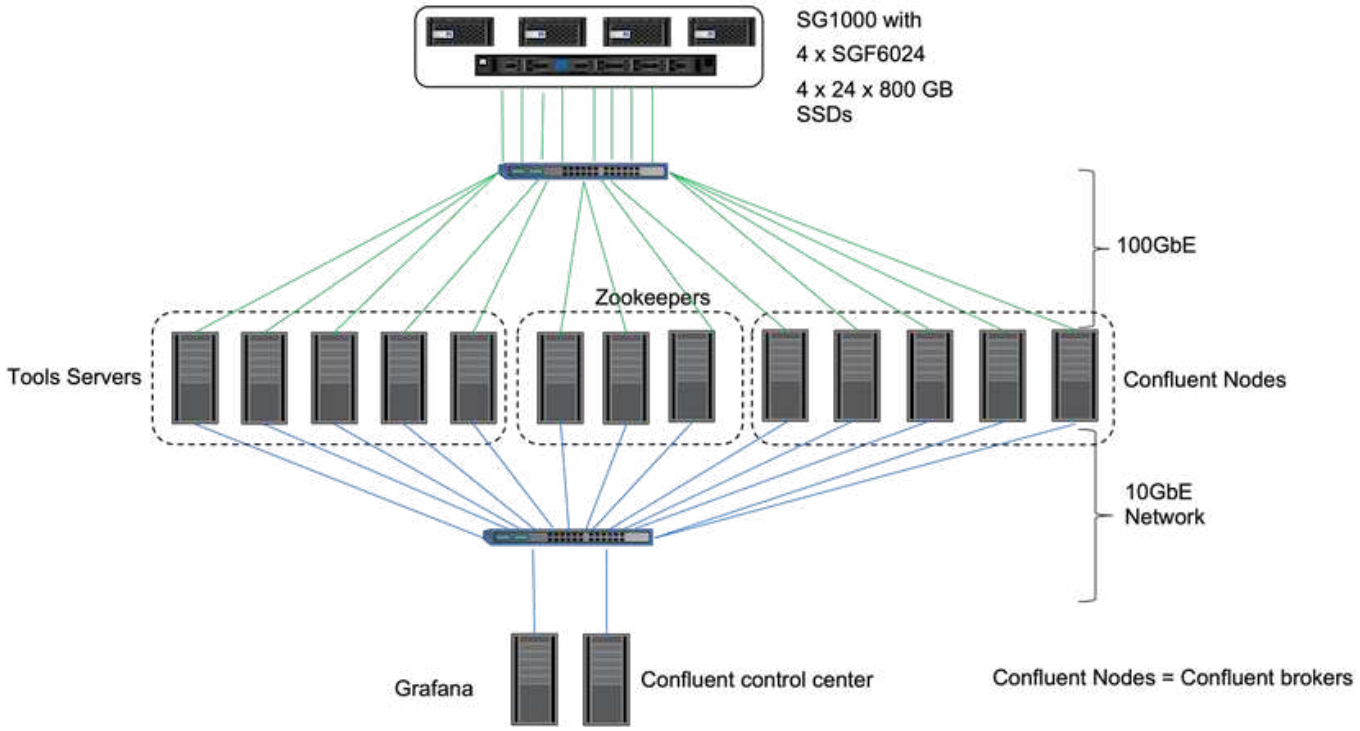
NetApp StorageGRID で、 Conluent Platform 6.2 の階層型ストレージを使用して検証を実施しました。ネットアップと流暢なチームがこの検証に協力し、検証に必要なテストケースを実施しました。

競合するプラットフォームの設定

検証には次のセットアップを使用しました。

検証には、 3 台の zookeepers 、 5 台のブローカー、 5 台のテストスクリプトを実行するサーバ、 256GB の RAM を搭載した tools サーバ、 16 個の CPU を使用しました。ネットアップストレージの場合は、 4 つの SGF6024 を搭載した SG1000 ロードバランサで StorageGRID を使用しました。ストレージとブローカーは、 100GbE 接続経由で接続されています。

次の図に、流暢な検証に使用される設定のネットワークトポロジを示します。



ツールサーバは、要求をノードに送信するアプリケーションクライアントとして機能します。

競合する階層型ストレージ構成

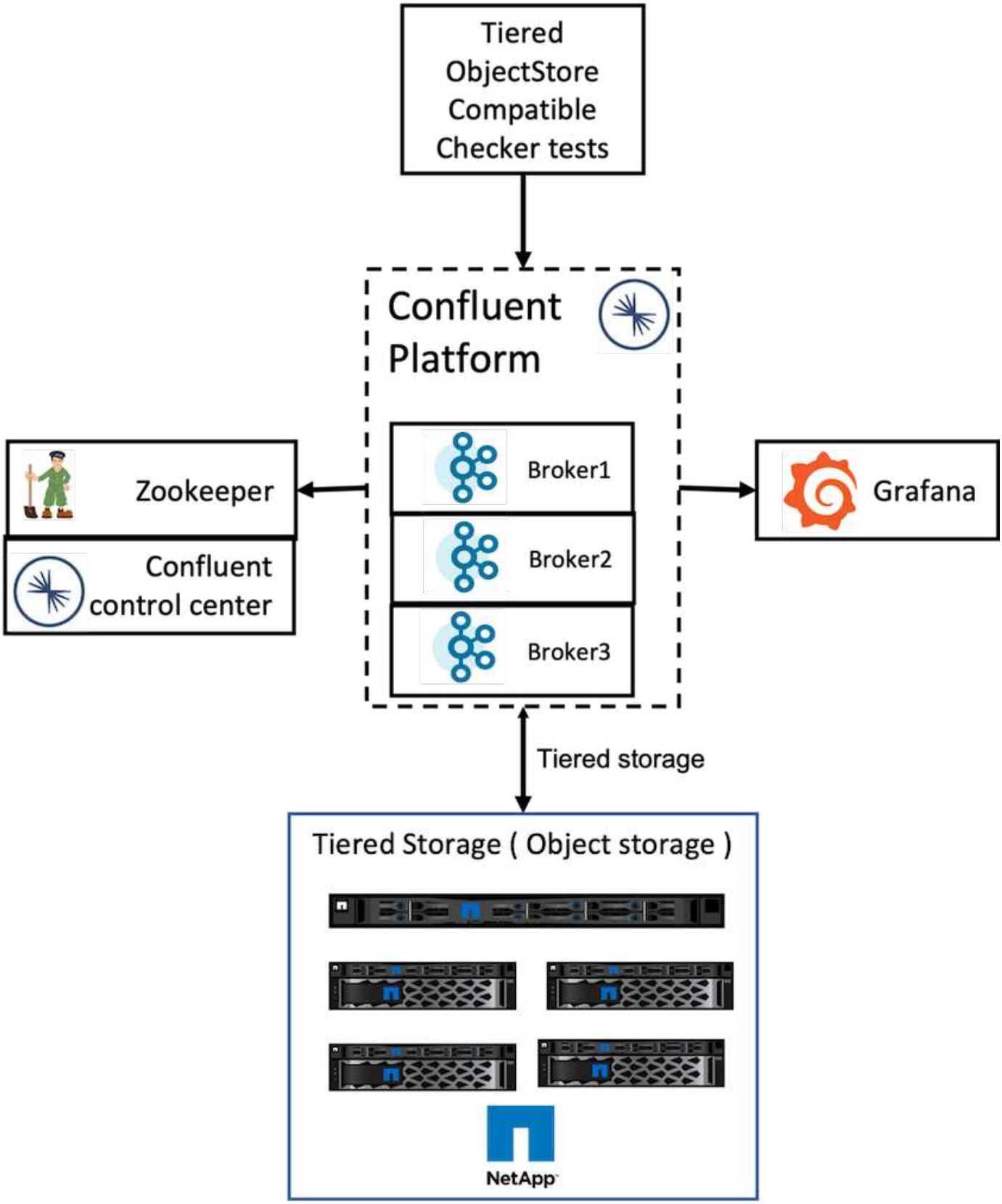
階層化ストレージの構成には、 Kafka に次のパラメータが必要です。

```
Confluent.tier.archiver.num.threads=16
confluent.tier.fetcher.num.threads=32
confluent.tier.enable=true
confluent.tier.feature=true
confluent.tier.backend=S3
confluent.tier.s3.bucket=kafkasgdbucket1-2
confluent.tier.s3.region=us-west-2
confluent.tier.s3.cred.file.path=/data/kafka/.ssh/credentials
confluent.tier.s3.aws.endpoint.override=http://kafkasgd.rtpppe.netapp.com:
10444/
confluent.tier.s3.force.path.style.access=true
```

検証 StorageGRID には HTTP プロトコルを使用しましたが、 HTTPS も使用できます。アクセスキーとシークレットキーは、「 confliclus.tir.s3.cred.file.path 」パラメータで指定したファイル名に格納されます。

ネットアップオブジェクトストレージ - StorageGRID

単一サイト構成を StorageGRID で検証用に設定しました。



検証テスト

以下の 5 つの検証ケースを完了しました。これらのテストは、 Trogdor フレームワークで実行されます。最初の 2 つは機能テストで、残りの 3 つはパフォーマンステストです。

オブジェクトストアの正確性テスト

このテストでは、階層化ストレージのニーズに応じて、オブジェクトストア API のすべての基本的な処理（GET / PUT / DELETE など）が適切に機能するかどうかを確認します。これは、すべてのオブジェクトストアサービスが次のテストよりも先に実施されることを想定した基本的なテストです。合格または不合格の自己主張的なテストです。

階層化機能の正確性テスト

このテストでは ' エンド・ツー・エンドの階層型ストレージ機能が ' 合格または不合格のアサート型テストで適切に機能するかどうかを判断しますテストでは、デフォルトで階層化が有効になっており、ホットセットサイズが大幅に縮小されたテストトピックが作成されます。新しく作成されたテストトピックへのイベントストリームが生成され、ブローカーがセグメントをオブジェクトストアにアーカイブするのを待機し、イベントストリームを消費して、消費されたストリームが生成されたストリームと一致することを検証します。イベントストリームに生成されるメッセージの数は設定可能で、テストのニーズに応じてユーザが十分な大きさのワークロードを生成できます。ホットセットのサイズを小さくすることで、消費者がアクティブなセグメントの外部でフェッチしたファイルはオブジェクトストアからのみ提供されます。これにより、オブジェクトストアの読み取りの正確性をテストできます。このテストは、オブジェクトストアフォールト挿入の有無にかかわらず実施しました。StorageGRID のいずれかのノードでサービスマネージャサービスを停止し、エンドツーエンド機能がオブジェクトストレージで機能することを検証することで、ノード障害をシミュレートしました。

ティアフェッチベンチマーク

このテストでは、階層型オブジェクトストレージの読み取りパフォーマンスを検証し、ベンチマークによって生成されたセグメントからの負荷が大きい範囲での読み取り要求のフェッチをチェックしました。このベンチマークでは、 Conluent 社は階層フェッチ要求に対応するカスタムクライアントを開発しました。

ワークロードベンチマークを消費

このテストでは、セグメントをアーカイブすることにより、オブジェクトストアへの書き込みワークロードを間接的に生成しました。コンシューマグループがセグメントを取得すると、読み取りワークロード（セグメント読み取り）がオブジェクトストレージから生成されました。このワークロードはテストスクリプトで生成されました。このテストでは、並列スレッドでのオブジェクトストレージの読み取りと書き込みのパフォーマンスをチェックしました。階層化機能の正確性テストと同様に、オブジェクトストアフォールト挿入を使用したテストと使用しなかったテストを実施しました。

保存ワークロードベンチマーク

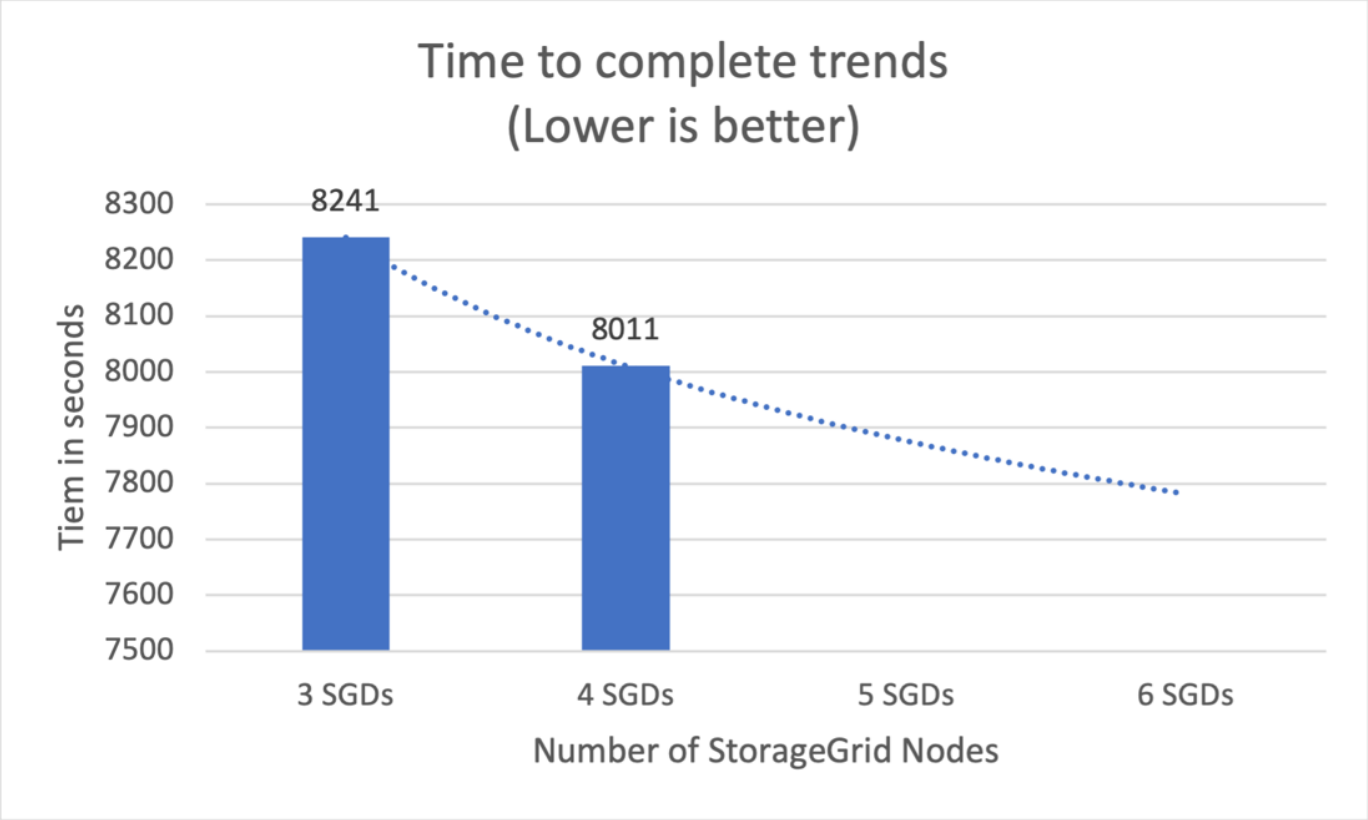
このテストでは、トピックの保持ワークロードが多い場合のオブジェクトストアの削除パフォーマンスを確認しました。保持ワークロードは、テストトピックと並行して多数のメッセージを生成するテストスクリプトを使用して生成されました。テストトピックでは、サイズベースおよび時間ベースの強力な保持設定を使用してイベントストリームをオブジェクトストアから継続的にパージするように設定しました。その後、セグメントがアーカイブされました。その結果、ブローカーによるオブジェクトストレージの削除や、オブジェクトストアの削除処理のパフォーマンス収集が行われ、大量の削除が発生していました。

拡張性を備えたパフォーマンステスト

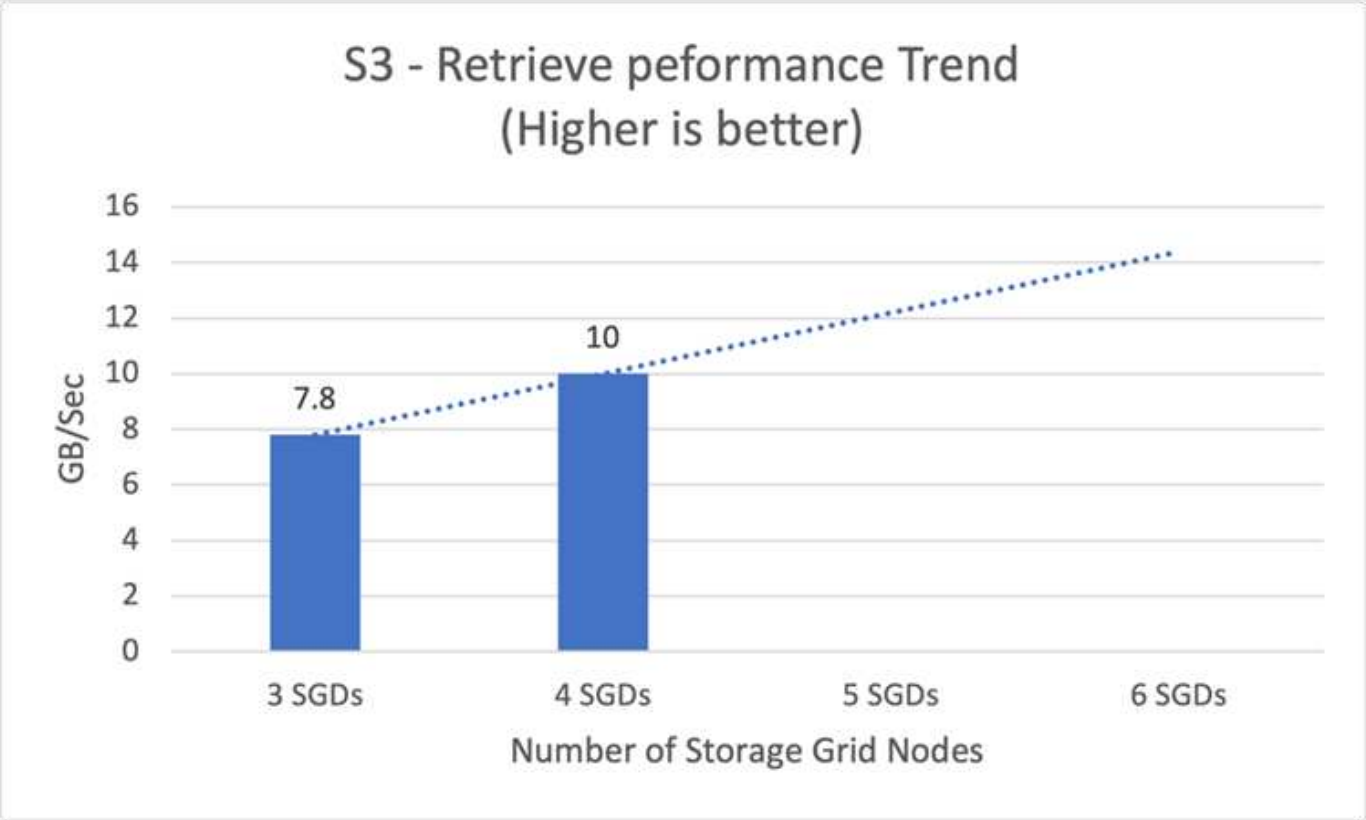
このティアストレージのテストでは、プロデューサーワークロードとコンシューマーワークロード向けに、 NetApp StorageGRID セットアップを使用して、 3 ノードから 4 ノードのノードを使用しました。テストによると、完了までの時間とパフォーマンス結果は StorageGRID ノードの数に直接比例しました。StorageGRID セットアップには、少

なくとも 3 つのノードが必要でした。

- ストレージノードの数が増えると、農産物と消費者の処理を完了するまでの時間が直線的に短くなりました。



- s3 読み出し処理のパフォーマンスは、 StorageGRID ノードの数に基づいてリニアに向上します。StorageGRID は、最大 200 個の StorgeGRID ノードをサポートします。



矛盾点 3 コネクタ

Amazon S3 Sink Connector は、 Apache Kafka のトピックから Avro 、 JSON 、またはバイト形式の S3 オブジェクトにデータをエクスポートします。Amazon S3 シンクコネクタは、 Kafka から定期的にデータをポーリングし、 S3 にアップロードします。ユーザは、各 Kafka パーティションのデータをチャンクに分割するために使用します。データのチャンクはそれぞれ S3 オブジェクトとして表されます。キー名は、トピック、 Kafka パーティション、およびこのデータチャンクの開始オフセットをエンコードします。

このセットアップでは、 Kafka s3 sink Connector を使用して、 Kafka のオブジェクトストレージのトピックの読み取りと書き込みを直接実行する方法を紹介します。このテストでは、スタンドアロンの流暢なクラスタを使用しましたが、このセットアップは分散クラスタに適用できます。

1. Conluent Kafka の Web サイトからダウンロードできます。
2. パッケージをサーバー上のフォルダに展開します。
3. 2 つの変数をエクスポートします。

```
Export CONFLUENT_HOME=/data/confluent/confluent-6.2.0
export PATH=$PATH:/data/confluent/confluent-6.2.0/bin
```

4. スタンドアロンの ConFluent Kafka セットアップの場合、クラスタは「 /tmp 」に一時的なルートフォルダを作成します。Zookeeper 、 Kafka 、スキーマレジストリ、 connect 、 ksql-server 、 とコントロール

センターのフォルダを作成し、それぞれの構成ファイルを「 $confliclus_home 」からコピーします。次の例を参照してください。

```
root@stlrx2540m1-108:~# ls -ltr /tmp/confluent.406980/
total 28
drwxr-xr-x 4 root root 4096 Oct 29 19:01 zookeeper
drwxr-xr-x 4 root root 4096 Oct 29 19:37 kafka
drwxr-xr-x 4 root root 4096 Oct 29 19:40 schema-registry
drwxr-xr-x 4 root root 4096 Oct 29 19:45 kafka-rest
drwxr-xr-x 4 root root 4096 Oct 29 19:47 connect
drwxr-xr-x 4 root root 4096 Oct 29 19:48 ksql-server
drwxr-xr-x 4 root root 4096 Oct 29 19:53 control-center
root@stlrx2540m1-108:~#
```

5. Zookeeper を構成します。デフォルトのパラメータを使用する場合は、何も変更する必要はありません。

```
root@stlrx2540m1-108:~# cat
/tmp/confluent.406980/zookeeper/zookeeper.properties  | grep -iv ^#
dataDir=/tmp/confluent.406980/zookeeper/data
clientPort=2181
maxClientCnxns=0
admin.enableServer=false
tickTime=2000
initLimit=5
syncLimit=2
server.179=controlcenter:2888:3888
root@stlrx2540m1-108:~#
```

上記の設定では、サーバを更新しました。xxx ’プロパティ。デフォルトでは、 Kafka リーダーの選択に 3 名の Zookeepers が必要です。

6. myid ファイルを tmp/conflicluent .406980/zookeeper /data に一意の ID で作成しました。

```
root@stlrx2540m1-108:~# cat /tmp/confluent.406980/zookeeper/data/myid
179
root@stlrx2540m1-108:~#
```

myid ファイルの最後の数の IP アドレスを使用しました。Kafka 、 connect 、 control-ccenter、 Kafka 、 Kafkakarest 、 ksql-server 、およびスキーマレジストリ設定。

7. Kafka サービスを開始します。

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  start
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
ZooKeeper is [UP]
Kafka is [UP]
Schema Registry is [UP]
Kafka REST is [UP]
Connect is [UP]
ksqlDB Server is [UP]
Control Center is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

構成ごとにログフォルダがあり、問題のトラブルシューティングに役立ちます。場合によっては、サービスの開始に時間がかかることがあります。すべてのサービスが稼働中であることを確認します。

8. 「 confliclue-hub 」を使用して Kafka connect をインストールします。

```
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# ./confluent-
hub install confluentinc/kafka-connect-s3:latest
The component can be installed in any of the following Confluent
Platform installations:
  1. /data/confluent/confluent-6.2.0 (based on $CONFLUENT_HOME)
  2. /data/confluent/confluent-6.2.0 (where this tool is installed)
Choose one of these to continue the installation (1-2): 1
Do you want to install this into /data/confluent/confluent-
6.2.0/share/confluent-hub-components? (yN) y

Component's license:
Confluent Community License
http://www.confluent.io/confluent-community-license
I agree to the software license agreement (yN) y
Downloading component Kafka Connect S3 10.0.3, provided by Confluent,
Inc. from Confluent Hub and installing into /data/confluent/confluent-
6.2.0/share/confluent-hub-components
Do you want to uninstall existing version 10.0.3? (yN) y
Detected Worker's configs:
  1. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  2. Standard: /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  3. Standard: /data/confluent/confluent-6.2.0/etc/schema-
```

```
registry/connect-avro-distributed.properties
  4. Standard: /data/confluent/confluent-6.2.0/etc/schema-
registry/connect-avro-standalone.properties
  5. Based on CONFLUENT_CURRENT:
/tmp/confluent.406980/connect/connect.properties
  6. Used by Connect process with PID 15904:
/tmp/confluent.406980/connect/connect.properties
Do you want to update all detected configs? (yN) y
Adding installation directory to plugin path in the following files:
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/kafka/connect-
standalone.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
distributed.properties
  /data/confluent/confluent-6.2.0/etc/schema-registry/connect-avro-
standalone.properties
  /tmp/confluent.406980/connect/connect.properties
  /tmp/confluent.406980/connect/connect.properties

Completed
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

また、「 conflicluent -hub install conflicentinc / Kafka-connect-s3 ： 10.0.3` を使用して、特定のバージョンをインストールすることもできます。

9. デフォルトでは、「 confluentinc - Kafka-connect-s3 」は「 /data/luent confin/conflicluent - 6.2.0/conflicluent -huber-components/conflicentinc - Kafka-connect-s3 」にインストールされています。
10. 新しい「 confluentinc - Kafka -connect-s3` でプラグインパスを更新します。

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/etc/kafka/connect-distributed.properties | grep plugin.path
#
plugin.path=/usr/local/share/java,/usr/local/share/kafka/plugins,/opt/co
nnectors,
plugin.path=/usr/share/java,/data/zookeeper/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components,/data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-s3
root@stlrx2540m1-108:~#
```

11. 流暢なサービスを停止し、再起動します。

```
confluent local services  stop
confluent local services  start
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin# confluent
local services  status
The local commands are intended for a single-node development
environment only,
NOT for production usage.

Using CONFLUENT_CURRENT: /tmp/confluent.406980
Connect is [UP]
Control Center is [UP]
Kafka is [UP]
Kafka REST is [UP]
ksqlDB Server is [UP]
Schema Registry is [UP]
ZooKeeper is [UP]
root@stlrx2540m1-108:/data/confluent/confluent-6.2.0/bin#
```

12. アクセス ID とシークレットキーを「 /root/.AWS/credentials 」ファイルに設定します。

```
root@stlrx2540m1-108:~# cat /root/.aws/credentials
[default]
aws_access_key_id = xxxxxxxxxxxx
aws_secret_access_key = xxxxxxxxxxxxxxxxxxxxxxxxx
root@stlrx2540m1-108:~#
```

13. バケットに到達できることを確認します。

```
root@stlrx2540m4-01:~# aws s3 –endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls kafkasgdbucket1-2
2021-10-29 21:04:18       1388 1
2021-10-29 21:04:20       1388 2
2021-10-29 21:04:22       1388 3
root@stlrx2540m4-01:~#
```

14. s3 およびバケット設定用の s3-sink プロパティファイルを設定します。

```
root@stlrx2540m1-108:~# cat /data/confluent/confluent-
6.2.0/share/confluent-hub-components/confluentinc-kafka-connect-
s3/etc/quickstart-s3.properties | grep -v ^#
name=s3-sink
connector.class=io.confluent.connect.s3.S3SinkConnector
tasks.max=1
topics=s3_testtopic
s3.region=us-west-2
s3.bucket.name=kafkasgdbucket1-2
store.url=http://kafkasgd.rtpppe.netapp.com:10444/
s3.part.size=5242880
flush.size=3
storage.class=io.confluent.connect.s3.storage.S3Storage
format.class=io.confluent.connect.s3.format.avro.AvroFormat
partitioner.class=io.confluent.connect.storage.partitioner.DefaultPartit
ioner
schema.compatibility=NONE
root@stlrx2540m1-108:~#
```

15. s3 バケットに数件のレコードをインポートします。

```
kafka-avro-console-producer --broker-list localhost:9092 --topic
s3_topic \
--property
value.schema='{"type":"record","name":"myrecord","fields":[{"name":"f1",
"type":"string"}]}'
{"f1": "value1"}
{"f1": "value2"}
{"f1": "value3"}
{"f1": "value4"}
{"f1": "value5"}
{"f1": "value6"}
{"f1": "value7"}
{"f1": "value8"}
{"f1": "value9"}
```

16. S3 シンクコネクタを取り付けます。

```
root@stlrx2540m1-108:~# confluent local services connect connector load
s3-sink  --config /data/confluent/confluent-6.2.0/share/confluent-hub-
components/confluentinc-kafka-connect-s3/etc/quickstart-s3.properties
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "config": {
    "connector.class": "io.confluent.connect.s3.S3SinkConnector",
    "flush.size": "3",
    "format.class": "io.confluent.connect.s3.format.avro.AvroFormat",
    "partitioner.class":
"io.confluent.connect.storage.partitioner.DefaultPartitioner",
    "s3.bucket.name": "kafkasgdbucket1-2",
    "s3.part.size": "5242880",
    "s3.region": "us-west-2",
    "schema.compatibility": "NONE",
    "storage.class": "io.confluent.connect.s3.storage.S3Storage",
    "store.url": "http://kafkasgd.rtpppe.netapp.com:10444/",
    "tasks.max": "1",
    "topics": "s3_testtopic",
    "name": "s3-sink"
  },
  "tasks": [],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

17. s3-sink のステータスを確認します。

```
root@stlrx2540m1-108:~# confluent local services connect connector
status s3-sink
The local commands are intended for a single-node development
environment only,
NOT for production usage.
https://docs.confluent.io/current/cli/index.html
{
  "name": "s3-sink",
  "connector": {
    "state": "RUNNING",
    "worker_id": "10.63.150.185:8083"
  },
  "tasks": [
    {
      "id": 0,
      "state": "RUNNING",
      "worker_id": "10.63.150.185:8083"
    }
  ],
  "type": "sink"
}
root@stlrx2540m1-108:~#
```

18. ログをチェックして、 s3-sink のトピックを受け入れる準備ができていることを確認します。

```
root@stlrx2540m1-108:~# confluent local services connect log
```

19. Kafka のトピックを確認してください。

```
kafka-topics --list --bootstrap-server localhost:9092
…
connect-configs
connect-offsets
connect-statuses
default_ksql_processing_log
s3_testtopic
s3_topic
s3_topic_new
root@stlrx2540m1-108:~#
```

20. s3 バケット内のオブジェクトを確認します。

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 ls --recursive kafkasgdbucket1-
2/topics/
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000003.avro
2021-10-29 21:24:00        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000006.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000009.avro
2021-10-29 21:24:08        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000012.avro
2021-10-29 21:24:09        213
topics/s3_testtopic/partition=0/s3_testtopic+0+0000000015.avro
root@stlrx2540m1-108:~#
```

21. 内容を確認するには、次のコマンドを実行して、 S3 からローカルファイルシステムに各ファイルをコピーします。

```
root@stlrx2540m1-108:~# aws s3 --endpoint-url
http://kafkasgd.rtpppe.netapp.com:10444 cp s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro
tes.avro
download: s3://kafkasgdbucket1-
2/topics/s3_testtopic/partition=0/s3_testtopic+0+0000000000.avro to
./tes.avro
root@stlrx2540m1-108:~#
```

22. レコードを印刷するには、 avro-tools-1.11.0.1.jar を使用します（『』で入手できます） "Apache アーカイブ"）。

```
root@stlrx2540m1-108:~# java -jar /usr/src/avro-tools-1.11.0.1.jar
tojson tes.avro
21/10/30 00:20:24 WARN util.NativeCodeLoader: Unable to load native-
hadoop library for your platform... using builtin-java classes where
applicable
{"f1":"value1"}
{"f1":"value2"}
{"f1":"value3"}
root@stlrx2540m1-108:~#
```

競合する自己バランシングクラスタ

Kafka クラスタを以前に管理していたことがある場合、パーティションを別のブローカーに手動で再割り当てすることで、ワークロードがクラスタ全体に分散されるようにするという問題に慣れている方がよいでしょう。Kafka を大規模に導入している組織では、大容量のデータを変更するのが難しく、面倒でリスクが伴う可能性があります。特に、ミッションクリティカルなアプリケーションをクラスタの上に構築する場合、リスクが高くなります。しかし、 Kafka を使用するケースが最小であっても、処理に時間がかかり、人為的ミスが発生しやすくなります。

このラボでは、流暢な自己分散クラスタ機能をテストし、クラスタトポロジの変更や不均衡な負荷に基づいてリバランシングを自動化しました。一括リバランシングテストは、ノード障害や拡張ノードでブローカー間のデータのリバランシングが必要になった場合に、新しいブローカーを追加する時間を測定するのに役立ちます。従来の Kafka 構成では、クラスタの拡張に合わせてデータの再分散量が増える一方で、階層型ストレージでは少量のデータしかリバランシングされません。この検証に基づき、階層型ストレージのリバランシングには数秒から数分かかりますが、従来の Kafka アーキテクチャでは、クラスタの拡張に伴ってリニアに拡張されます。

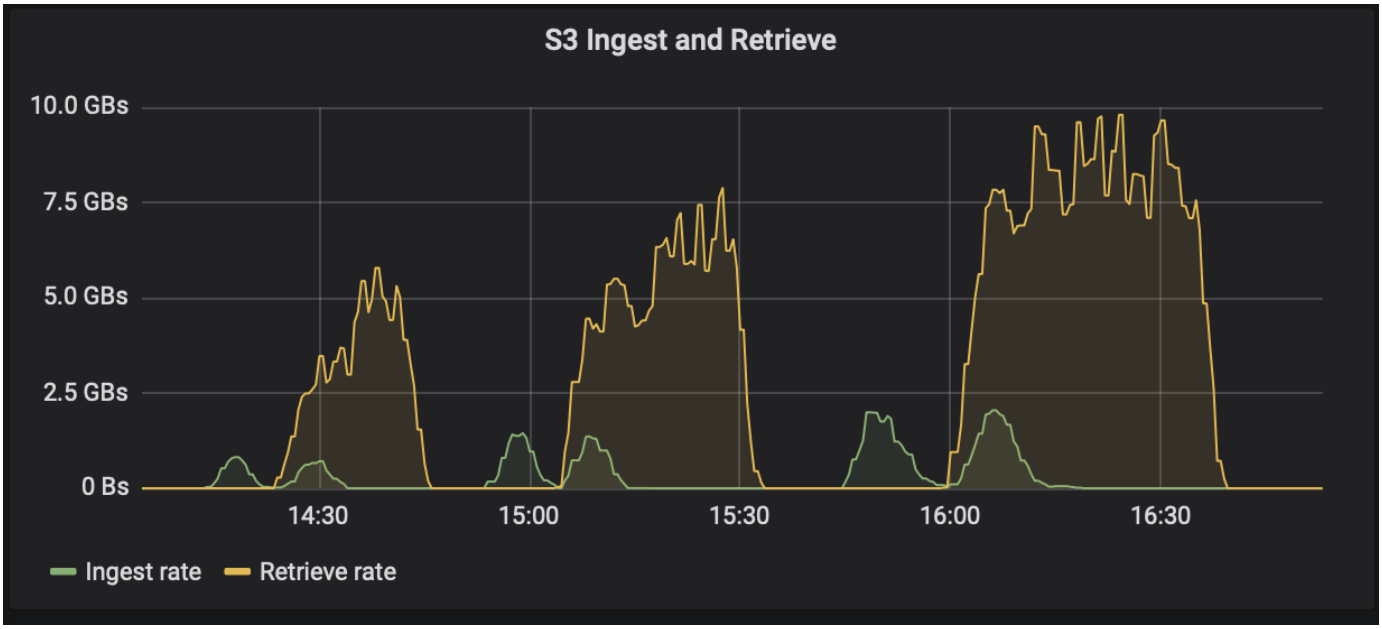
セルフバランシングクラスタでは、パーティションの再調整が完全に自動化され、 Kafka のスループットが最適化され、ブローカーの拡張が高速化され、大規模なクラスタを実行する際の運用上の負担が軽減されます。安定した状態では、自己バランシングクラスタがブローカー間のデータのスキューを監視し、パーティションを継続的に再割り当てしてクラスタのパフォーマンスを最適化します。プラットフォームをスケールアップまたはスケールダウンすると、新しいブローカーが存在することを自己分散クラスタが自動的に認識したり、古いブローカーが削除されたことを確認したり、後続のパーティションの再割り当てをトリガーしたりします。これにより、ブローカーの追加や運用停止が容易になり、 Kafka クラスタの柔軟性が根本的に向上します。これらの利点は、手動操作、複雑な計算、またはパーティションの再割り当てが一般的に発生する人的ミスのリスクを必要としないことです。その結果、データのリバランシングが完了するまでの時間が大幅に短縮され、クラスタを常時監視するのではなく、価値の高いイベントストリーミングプロジェクトに集中できます。

ベストプラクティスのガイドライン

このセクションでは、この認定資格から得られた教訓について説明します。

- 検証に基づくと、 S3 オブジェクトストレージにはデータを格納するのがベストプラクティスです。
- 高スループットの SAN （特に FC ）を使用して、ブローカーのホットデータやローカルディスクを保持できます。これは、階層型ストレージの構成が流暢であるためです。 ブローカーデータディレクトリに格納されているデータのサイズは、オブジェクトストレージにデータが移動される際のセグメントサイズと保持時間に基づいています。
- オブジェクトストレージは、セグメントの方がパフォーマンスに優れています。バイト数が多い場合は 512 MB をテストしました。
- Kafka では、トピックに対して生成される各レコードのキーまたは値の長さ（バイト単位）は、「 length.key.value 」パラメータによって制御されます。StorageGRID では、 S3 オブジェクトの取り込みと読み出しのパフォーマンスがより高い値に引き上げられました。たとえば、 512 バイトが 5.8GBps の読み出しを提供し、 1024 バイトが 7.5GBps の s3 読み出しを提供し、 2048 バイトが 10Gbps 近く提供します。

次の図に、「 length.key.value 」に基づいた S3 オブジェクトの取り込みと読み出しを示します。



- * Kafka のチューニング。 * 階層型ストレージのパフォーマンスを向上させるには、TierFetcherNumThreads と TierArchiverNumThreads を増やします。一般的なガイドラインとして、TierFetcherNumThreads を増やして物理 CPU コア数に合わせ、 TierArchiverNumThreads を CPU コア数の半分に増やします。たとえば、サーバープロパティで、 8 つの物理コアを持つマシンがある場合、conflicent.tier.fetcher.threads=8 と、 conflicent.tier.Archiver .num.threads=4 を設定します。
- * トピックが削除されるまでの時間です。 * トピックが削除されると、オブジェクトストレージ内のログセグメントファイルの削除はすぐには開始されません。デフォルト値の 3 時間を指定した時間間隔が設定されているため、この時間が経過するとファイルが削除されます。この間隔の値を変更するには、設定 confluent.tier.topic.delete.check.interval.ms を変更します。トピックまたはクラスタを削除する場合は、それぞれのバケット内のオブジェクトを手動で削除することもできます。
- * 階層型ストレージの内部トピックに関する ACL 。 * オンプレミス環境に推奨されるベストプラクティスは、階層型ストレージに使用される内部トピックについて、 ACL 承認者を有効にすることです。このデータへのアクセスをブローカーユーザのみに制限するには、 ACL ルールを設定してください。これにより、内部トピックが保護され、階層化されたストレージデータおよびメタデータへの不正アクセスを防止できます。

```
kafka-acls --bootstrap-server localhost:9092 --command-config adminclient-
configs.conf \
--add --allow-principal User:<kafka> --operation All --topic "_confluent-
tier-state"
```

ユーザ「 <Kafka> 」を、導入環境の実際のブローカプリンシパルに置き換えます。

たとえば ' コマンド confliclus-tier-state は ' 階層型ストレージの内部トピックに ACL を設定します現在、階層化ストレージに関連する内部トピックは 1 つだけです。この例では、内部トピックのすべての処理にプリンシパル Kafka 権限を提供する ACL を作成しています。

サイジング

Kafka サイジングは、シンプル、きめ細かい、リバース、およびパーティションの 4 つ

の構成モードで実行できます。

シンプル

シンプルモードは、 Apache Kafka を初めて使用するユーザや初期状態のユースケースに適しています。このモードでは、スループット MBps 、読み取りファンアウト、保持、リソース利用率（デフォルトは 60% ）などの要件を指定します。オンプレミス（ベアメタル、 VMware 、 Kubernetes 、 OpenStack ）やクラウドなどの環境にも移行できます。Kafka クラスタのサイジングは、この情報に基づいて、ブローカーに必要なサーバ数、 Zookeeper 、 Apache Kafka Connect Workers 、スキーマレジストリ、 REST プロキシ、 ksqlDB 、および Conluent コントロールセンターを提供します。

階層型ストレージの場合、 Kafka クラスタのサイジングのためのきめ細かい構成モードを検討してください。Granular モードは、経験豊富な Apache Kafka ユーザや明確に定義されたユースケースに適しています。このセクションでは、プロデューサ、ストリームプロセッサ、およびコンシューマのサイジングについて説明します。

プロデューサー

Apache Kafka の開発者（ネイティブクライアント、 REST プロキシ、 Kafka コネクタなど）については、次の情報を参照してください。

- * 名前。 * Spark 。
- * 開発者の種類。 * アプリケーションまたはサービス、プロキシ (REST 、 MQTT 、その他) 、および既存のデータベース (RDBMS, NoSQL 、その他) 。「わからない」を選択することもできます。
- * 平均スループット。 * 1 秒あたりのイベント数（ 1 、 000 、 000 など）。
- * 最大スループット。 * 1 秒あたりのイベント数（ 4 、 000 、 000 など）。
- * 平均メッセージサイズ。 * バイト単位、非圧縮（最大 1MB 、 1000 など）。
- * メッセージ形式。 * Avro 、 JSON 、プロトコルバッファ、バイナリ、テキスト、 「わからない」など。
- * 複製係数 * オプションは 1 、 2 、 3 （流暢な推奨）、 4 、 5 です。 または 6.
- * 保持時間。 * 1 日（例：）。Apache Kafka にデータを保存しておく期間を教えてください。無期限には、任意の単位で -1 を入力します。無制限の保持期間を 10 年と想定しています。
- [ブローカー数を減らし、 Infinite Storage を許可するための階層化ストレージを有効にする] のチェックボックスをオンにします。
- 階層型ストレージが有効になっている場合は ' リテンションフィールドによって ' ブローカにローカルに保存されるデータのホットセットが制御されますアーカイブ保持フィールドは、アーカイブオブジェクトストレージにデータを格納する期間を制御します。
- * アーカイブ・ストレージの保存期間 * 1 年（例：）データをアーカイブストレージに保存する期間無期限の任意の単位を指定して -1 を入力します。無制限の保持期間を 10 年と想定しています。
- * 成長乗数 * 1 （例：）。このパラメータの値が現在のスループットに基づく場合は、 1 に設定します。追加の増加に基づいてサイズを決定するには、このパラメータに増加率を設定します。
- * プロデューサーインスタンスの数。 * 10 （例：）実行されるプロデューサーインスタンスの数はいくつですか？この値は、 CPU 負荷をサイジング計算に含めるために必要です。空白の値は、 CPU 負荷が計算に組み込まれていないことを示します。

この入力例に基づいて、サイジングはプロデューサーに次のように影響します。

- 非圧縮バイト単位の平均スループット： 1Gbps圧縮されていないバイト数のピークスループット： 4Gbps圧縮されたバイト数の平均スループット： 400MBps 。圧縮バイトの最大スループット： 1.6GBps 。デフォルトの 60% の圧縮率に基づいています（この値は変更できます）。
  - 必要なホットセットストレージの合計数 :31 、 104TB (レプリケーションを含む) 、圧縮。必要なオフブローカーアーカイブストレージの合計： 378 、 432TB 、圧縮使用 "https://fusion.netapp.com" StorageGRID のサイジングの場合：

Stream Processors は、 Apache Kafka のデータを使用し、 Apache Kafka に生成するアプリケーションまたはサービスを記述する必要があります。ほとんどの場合、これらは ksqlDB ストリームまたは Kafka ストリームで構築されます。

- * 名前。 * Spark streamer 。
- * 処理時間。 * このプロセッサーは 1 つのメッセージを処理するのにどれくらいかかりますか？
  - 1 ミリ秒（シンプルでステートレスな変換）（例）、 10 ミリ秒（ステートフルなメモリ内動作）
  - 100ms （ステートフルネットワークまたはディスク処理）、 1000ms （サードパーティ製 REST コール）
  - このパラメータをベンチマークして、その所要時間を正確に把握しました。
- * 出力保持 * 1 日（例）ストリームプロセッサは Apache Kafka に出力を返します。この出力データを Apache Kafka にどれくらいの期間保存しますか？無期限の任意の単位を指定して -1 を入力します。
- ［ ブローカー数を減らし、 Infinite Storage を許可するには、 ［ 階層型ストレージを有効にする ］ チェックボックスをオンにします。
- * アーカイブ・ストレージの保存期間 * 1 年（例：）データをアーカイブストレージに保存する期間無期限の任意の単位を指定して -1 を入力します。無制限の保持期間を 10 年と想定しています。
- * 出力パススルー率。 * 100 （例：）ストリームプロセッサは Apache Kafka に出力を返します。Apache Kafka に出力されるインバウンドスループットの割合を教えてください。たとえば、インバウンドスループットが 20Mbps で、この値が 10 の場合、出力スループットは 2Mbps になります。
- この機能はどのアプリケーションから読み取られますか。「 Spark 」を選択すると、販売担当者タイプに基づいたサイジングで使用された名前になります。上記の入力に基づいて、ストリームプロセッサインスタンスとトピックパーティションの推定にサイジングを行うと、次のような効果が期待できます。
- このストリームプロセッサアプリケーションには、次の数のインスタンスが必要です。受信するトピックでは、多くのパーティションが必要になる場合があります。このパラメータを確認するには、 [流暢] に連絡してください
  - 平均スループットで 1 、 000 、増加率なし
  - 4 、 000 ：増加率のないピークスループット
  - 1 、 000 ：平均スループットと増加率
  - 4 、 000 ：最大スループットで増加率

消費者

Apache Kafka のデータを利用していて、 Apache Kafka にデータを生成していないアプリケーションやサービスについて説明してください。たとえば、ネイティブのクライアントや Kafka Connector などです。

- * 名前 . * Spark consumer 。
- * 処理時間。 * この消費者は、 1 つのメッセージを処理するのにどれくらいの時間がかかりますか。

- 1 ミリ秒（シンプルでステートレスなロギングなどのタスク）
- 10 ミリ秒（データストアへの高速書き込み）
- 100 ミリ秒（データストアへの書き込み速度が遅い）
- 1000 ミリ秒（サードパーティの REST コール）
- その他のベンチマークされたプロセスの中には、既知の期間があります。

- * 消費者タイプ。 * アプリケーション、プロキシ、シンクを既存のデータストア（ RDBMS 、 NoSQL など）に。
- この機能はどのアプリケーションから読み取られますか。このパラメータは、以前に決定したプロデューサーおよびストリームのサイジングを使用して接続します。

上記の入力に基づいて、コンシューマインスタンスとトピックパーティションの推定サイズを決定する必要があります。コンシューマアプリケーションには、次の数のインスタンスが必要です。

- 平均スループットで 2 、 000 、増加率はゼロ
- 8 、 000 ：ピークスループットで、増加率はゼロです
- 平均スループットで 2 、 000 、増加率も含まれます
- 8 、 000 ：最大スループット。増大の乗数も含まれます

受信トピックでは、この数のパーティションも必要になる場合があります。確認のため、流暢な連絡をします。

生産者、ストリームプロセッサ、および消費者の要件に加えて、次の追加要件を提供する必要があります。

- * 再構築時間。 * 例： 4 時間。Apache Kafka ブローカーホストで障害が発生すると、そのデータは失われ、障害が発生したホストと交換するために新しいホストをプロビジョニングすると、この新しいホストをどのくらいの速さで再構築する必要がありますか？値が不明な場合は、このパラメータを空白のままにします。
- * リソース使用率目標（パーセンテージ）。 * 例： 60 。平均スループット中にホストをどの程度使用しますか？Conluent の自己バランシングクラスタを使用していない場合は、 60% の使用率を推奨します。この場合、使用率が高くなります。

環境の説明

- * どの環境でクラスターを実行しますか？ * Amazon Web Services 、 Microsoft Azure 、 Google クラウドプラットフォーム、オンプレミスのベアメタル、 VMware オンプレミス、 OpenStack をオンプレミスで運用するのか、 Kubernates をオンプレミスで運用するのか
- * ホストの詳細。 * コアの数： 48 （例：）、ネットワークカードのタイプ（ 10GbE 、 40GbE 、 16GbE 、 1GbE 、またはその他のタイプ）。
- * ストレージボリューム。 * ホスト： 12 （例：）。ホストあたりのハードドライブまたは SSD の数はいくつですか。競合するホストごとに 12 台のハードドライブを推奨します。
- * ストレージ容量 / ボリューム（ GB 単位）。 * 1000 （例：）。1 つのボリュームストアでギガバイト単位のストレージ容量はどれくらいですか？競合する場合は 1TB のディスクを使用します。
- * ストレージ構成 * ストレージ・ボリュームの構成方法競合製品は、 Conluent のすべての機能を活用するために RAID10 を推奨しています。JBOD 、 SAN 、 RAID 1 、 RAID 0 、 RAID 5 、 その他のタイプもサポートされています。

- * 単一ボリュームのスループット（ Mbps ）。 * 125 （例：）1 つのストレージボリュームで 1 秒あたりのメガバイト数で読み取りまたは書き込みを行うことができる速度はどれくらいですか。競合するハードディスクドライブは、通常 125 Mbps のスループットを持つ標準ハードディスクドライブをお勧めします。
- * メモリ容量（ GB ）。 * 64 （例）。

環境変数を決定したら、 Size my Cluster （マイクラスタのサイズ）を選択します。前述の例に基づいて、Con裕福 な Kafka のサイジングを決定しました。

- * Apache Kafka * Broker count ： 22 。クラスタはストレージバウンドです。階層型ストレージを有効にして、ホスト数を減らし、ストレージを無制限にすることを検討してください。
- * Apache ZooKeeper. * Count ： 5 ； Apache Kafka Connect Workers ： Count ： 2 ； Schema Registry ： Count ： 2 ； REST Proxy ： Count ： 2 ； ksqlDB ： Count ： 2 ； Conluent Control Center ： Count ： 1 。

ユースケースを考慮せずに、プラットフォームチームにリバースモードを使用する。パーティションモードを使用して、 1 つのトピックに必要なパーティションの数を計算します。を参照してください https://eventsizer.io リバースモードとパーティションモードに基づいたサイジングの場合

まとめ

このドキュメントでは、検証テスト、階層型ストレージのパフォーマンス結果、調整、S3 コネクタの堪能、セルフバランシング機能など、ネットアップストレージでの Conluent Tiered Storage の使用に関するベストプラクティスを紹介しています。ILM ポリシー、検証のための複数のパフォーマンステストと業界標準の S3 API を使用した流暢なパフォーマンスを考慮した場合、 NetApp StorageGRID オブジェクトストレージは流暢な階層化ストレージに最適な選択肢です。

追加情報の参照先

このドキュメントに記載されている情報の詳細については、以下のドキュメントや Web サイトを参照してください。

- Apache Kafka とは何ですか

  "https://www.confluent.io/what-is-apache-kafka/"

- ネットアップの製品マニュアル

  "https://www.netapp.com/support-and-training/documentation/"

- S3 シンクパラメータの詳細

  "https://docs.confluent.io/kafka-connect-s3-sink/current/configuration_options.html#s3-configuration-options"

- Apache Kafka です

  "https://en.wikipedia.org/wiki/Apache_Kafka"

- Conluent Platform の無限のストレージ

  "https://www.confluent.io/blog/infinite-kafka-storage-in-confluent-platform/"

- 競合する階層型ストレージ - ベストプラクティスとサイジング

  "https://docs.confluent.io/platform/current/kafka/tiered-storage.html#best-practices-and-recommendations"

- Conluent Platform 用の Amazon S3 シンクコネクタ

  "https://docs.confluent.io/kafka-connect-s3-sink/current/overview.html"

- Kafka のサイジング

  "https://eventsizer.io"

- StorageGRID のサイジング

  "https://fusion.netapp.com/"

- Kafka のユースケース

  "https://kafka.apache.org/uses"

- 統合されたプラットフォーム 6.0 の自律分散 Kafka クラスター

  "https://www.confluent.io/blog/self-balancing-kafka-clusters-in-confluent-platform-6-0/"

  "https://www.confluent.io/blog/confluent-platform-6-0-delivers-the-most-powerful-event-streaming-platform-to-date/"

著作権に関する情報

Copyright © 2024 NetApp, Inc. All Rights Reserved. Printed in the U.S.このドキュメントは著作権によって保護されています。著作権所有者の書面による事前承諾がある場合を除き、画像媒体、電子媒体、および写真複写、記録媒体、テープ媒体、電子検索システムへの組み込みを含む機械媒体など、いかなる形式および方法による複製も禁止します。

ネットアップの著作物から派生したソフトウェアは、次に示す使用許諾条項および免責条項の対象となります。

このソフトウェアは、ネットアップによって「現状のまま」提供されています。ネットアップは明示的な保証、または商品性および特定目的に対する適合性の暗示的保証を含み、かつこれに限定されないいかなる暗示的な保証も行いません。ネットアップは、代替品または代替サービスの調達、使用不能、データ損失、利益損失、業務中断を含み、かつこれに限定されない、このソフトウェアの使用により生じたすべての直接的損害、間接的損害、偶発的損害、特別損害、懲罰的損害、必然的損害の発生に対して、損失の発生の可能性が通知されていたとしても、その発生理由、根拠とする責任論、契約の有無、厳格責任、不法行為（過失またはそうでない場合を含む）にかかわらず、一切の責任を負いません。

ネットアップは、ここに記載されているすべての製品に対する変更を随時、予告なく行う権利を保有します。ネットアップによる明示的な書面による合意がある場合を除き、ここに記載されている製品の使用により生じる責任および義務に対して、ネットアップは責任を負いません。この製品の使用または購入は、ネットアップの特許権、商標権、または他の知的所有権に基づくライセンスの供与とはみなされません。

このマニュアルに記載されている製品は、1つ以上の米国特許、その他の国の特許、および出願中の特許によって保護されている場合があります。

権利の制限について：政府による使用、複製、開示は、DFARS 252.227-7013（2014年2月）およびFAR 5252.227-19（2007年12月）のRights in Technical Data -Noncommercial Items（技術データ - 非商用品目に関する諸権利）条項の(b)(3)項、に規定された制限が適用されます。

本書に含まれるデータは商用製品および / または商用サービス（FAR 2.101の定義に基づく）に関係し、データの所有権はNetApp, Inc.にあります。本契約に基づき提供されるすべてのネットアップの技術データおよびコンピュータ ソフトウェアは、商用目的であり、私費のみで開発されたものです。米国政府は本データに対し、非独占的かつ移転およびサブライセンス不可で、全世界を対象とする取り消し不能の制限付き使用権を有し、本データの提供の根拠となった米国政府契約に関連し、当該契約の裏付けとする場合にのみ本データを使用できます。前述の場合を除き、NetApp, Inc.の書面による許可を事前に得ることなく、本データを使用、開示、転載、改変するほか、上演または展示することはできません。国防総省にかかる米国政府のデータ使用権については、DFARS 252.227-7015(b)項（2014年2月）で定められた権利のみが認められます。

商標に関する情報

NetApp、NetAppのロゴ、http://www.netapp.com/TMに記載されているマークは、NetApp, Inc.の商標です。その他の会社名と製品名は、それを所有する各社の商標である場合があります。